are comfortable with the simple implementation we'll use here, you can use the pixel version of the game and upgrade the Dist-DQN to use convolutional layers.

7.4.1. Representing a probability distribution in Python

If you didn't read the optional section 7.3, the only important thing you missed is that instead of using a neural network to represent a Q function, $Q_{\pi}(s,a)$, that returns a single Q value, we can instead denote a value distribution, $Z_{\pi}(s,a)$, that represents a random variable of Q values given a state-action pair. This probabilistic formalism subsumes the deterministic algorithms we've been using in prior chapters, since a deterministic outcome can always be represented by a *degenerate* probability distribution (figure 7.8), where all the probability is assigned to a single outcome.

Figure 7.8. This is a degenerate distribution, since all the possible values are assigned a probability of 0 except for one value. The outcome values that are not assigned 0 probability are called the probability distribution's support. The degenerate distribution has a support of 1 element (in this case, the value 0).